

Rent & Flatmate Finder

Objective:

- Renting a room involves more than just price, it requires finding someone whose expectations on location and budget align. Build a platform where owners list rooms and tenants create 'looking for room' profiles. The system uses an AI-powered compatibility engine to score and rank matches, enables real-time chat once interest is expressed, and notifies users when a strong match is found.

Scope of Work:

- **Input:** Room listing details, tenant profile (preferred location, budget range)
- **Output:** Ranked listings with AI compatibility scores, real-time chat, email notifications on key events
- Owner can register, log in, and post room listings: location, rent, available from date, room type, furnishing status, and photos
- Tenant can register, log in, and create a profile with preferred location, budget range, and move-in date
- Tenant can browse and filter listings by location and budget; listings ranked by compatibility score
- AI compatibility score computed using LLM: given tenant preferences and listing details, LLM returns a score (0 to 100) with a brief explanation of why they are a good or poor match
- Tenant can send an interest request to an owner; owner can accept or decline
- Once interest is accepted, both parties can chat in real time via WebSocket; messages persisted in database
- Email notification sent to owner when a tenant with a high compatibility score (e.g. above 80) expresses interest
- Email notification sent to tenant when owner accepts or declines their interest
- Owner can mark a listing as filled; filled listings are hidden from search results
- Admin can manage users, listings, and view platform activity

Technical Expectations:

- Backend API, Frontend, Database with role-based auth (tenant / owner / admin)
- LLM integration for compatibility scoring; score and explanation stored in DB, not recomputed on every request
- Real-time chat via WebSocket with message persistence
- Email notifications for interest events (any free tier email service)
- LLM failures must be handled gracefully. If the LLM is unavailable, fall back to a rule-based score

LLM Usage Guidance:

- **Compatibility prompt:** "Given this room listing: and this tenant profile: , compute a compatibility score from 0 to 100 based on budget and location match. Return JSON: { score: number, explanation: string }"
- Store the score and explanation against each tenant-listing pair in the database

Deliverables:

1. Zip file with complete source code

2. README with setup guide, .env.example, API docs, DB schema, and LLM prompt with example I/O
3. Hosted application URL (Vercel, Render, Railway, or similar)
4. System design write-up (800 words max) covering compatibility scoring design, LLM integration and fallback, chat implementation, and notification flow

Evaluation Focus:

- AI compatibility scoring quality and fallback handling
- Real-time chat implementation and message persistence
- Notification flow and email integration
- Database schema and data modelling
- API design and code structure
- Documentation